Pontifical Catholic University of Campinas
School of Exact, Environmental and Technology Sciences
Bachelor of Data Science and Artificial Intelligence

Association Rules Mining with the Apriori Algorithm on the Supermarket Dataset

Data preparation in Python, Apriori execution in Weka, and metric interpretation

STUDENT Miguel Santos da Silva

Campinas, 2026 — 1st Semester

Table of Contents

1. Introduction and Objective
2. Understanding the Supermarket Dataset
3. Data Preparation and Cleaning in Python
    - 3.1 Reading the .arff file and converting to DataFrame
    - 3.2 Removing department## columns
    - 3.3 Removing zero-support columns
    - 3.4 Exporting the cleaned dataset to CSV
4. Weka Export and CSV Format Considerations
5. Apriori Configuration in Weka
    - 5.1 First run: relaxed confidence
    - 5.2 Second run: Lift-focused search
6. In-Depth Analysis of Selected Rules
    - 6.1 Rule 1 – Vegetables imply fruit
    - 6.2 Rule 2 – Bread and cake imply milk and cream
    - 6.3 Rule 3 – The monthly stock-up purchase
7. Metric Interpretation: Support, Confidence, Lift and Leverage
    - 7.1 Leverage in practical terms
    - 7.2 Reference formulas
8. Business Discussion, Limitations and Best Practices
9. Conclusion

A. Appendix A – Annotated Python Code

B. Appendix B – Quick Reference for the Oral Defense

C. Appendix C – Extended Cleaning Logic

1. Introduction and Objective

This project focuses on the study of association rules derived from the supermarket dataset, a classic benchmark in market basket analysis. The central goal is to identify which products tend to appear together in transactions and, from those patterns, extract actionable insights for retail decision-making, merchandising, and marketing.

Beyond the algorithmic application itself, the work requires a complete analytical pipeline: loading the original file, cleaning unwanted columns, correctly exporting the data, running Apriori in Weka with varied parameter configurations, and finally interpreting the discovered rules in terms of purchase context and commercial intervention.

The expected outcome is twofold. On the technical side, the deliverable includes a Python notebook and a cleaned dataset. On the analytical side, the project demonstrates that the metrics and rules have been understood in terms of their practical meaning and potential business applications.

This report documents the entire process in a clear and structured manner, going beyond an operational summary. The objective is to present the work as a complete academic and professional record, suitable for portfolio use and oral defense.

2. Understanding the Supermarket Dataset

The supermarket dataset follows the typical format of transactional market data. Each row represents a purchase, and each column represents a product or product category. The values indicate whether a given item was present in that transaction, making it well-suited for association rule algorithms.

In datasets of this type, the initial read often yields many binary or near-binary columns. In some versions exported by Weka, the absence of an item appears as the symbol ?, while presence appears as t (true). This encoding requires careful preprocessing, as incorrect handling can distort the subsequent mining entirely.

The original dataset also includes structural columns such as numbered department entries. These columns, while part of the original schema, do not contribute to discovering associations between products and introduce unnecessary noise in the analysis.

3. Data Preparation and Cleaning in Python

3.1 Reading the .arff file and converting to DataFrame

The first step was to import the supermarket file in .arff format, which is native to the Weka ecosystem. The scipy library provides the loadarff function, which reads the file and returns its content as a structured array. The metadata returned alongside the data is discarded using the _ convention, since it is not used in this pipeline.

After loading, the data was converted to a NumPy array and then to a Pandas DataFrame. The dtype=str parameter was applied to ensure all values were treated as text, preventing unintended

automatic type conversions during filtering.

```
from scipy.io.arff import loadarff
import numpy as np
import pandas as pd

data, _ = loadarff('supermarket.arff')
df = pd.DataFrame(np.array(data), dtype=str)

print(f"Loaded: {df.shape[0]} rows, {df.shape[1]} columns")
```

3.2 Removing department## columns

Columns prefixed with department are structural metadata that do not represent purchasable items. They carry no analytical value for basket mining and only increase the number of attributes without adding information. The removal was automated using Pandas string filtering, avoiding any hardcoded column names and making the operation fully reproducible.

```
dept_cols = df.filter(like='department').columns
df = df.drop(columns=dept_cols)

print(f"Removed {len(dept_cols)} department columns. Remaining: {df.shape[1]}")
```

3.3 Removing zero-support columns

After removing the department columns, the next step was to identify products that never appeared in any transaction — columns where every value was ?. In data mining terms, these columns have support equal to zero. They produce no useful rules and only inflate the dimensionality of the dataset.

A boolean mask was used to identify such columns: the DataFrame was compared element-wise against ?, and the all() method returned True only for columns where every single value matched. An assertion validates the result before export.

```
empty_cols = df.columns[(df == '?').all()]
df = df.drop(columns=empty_cols)

print(f"Removed {len(empty_cols)} zero-support columns. Final shape: {df.shape}")
assert not (df == '?').all().any(), "Zero-support columns still present after
cleaning."
```

3.4 Exporting the cleaned dataset to CSV

Once cleaned, the dataset was exported to CSV for use in Weka. The index=False parameter is required: without it, Pandas writes its integer row index as an additional column, which Weka would interpret as a valid attribute and include in rule generation, introducing artificial associations with no business meaning.

```
output_path = 'supermarket_clean.csv'
df.to_csv(output_path, index=False)

print(f"Exported to {output_path} — {df.shape[0]} rows, {df.shape[1]} columns")
```

4. Weka Export and CSV Format Considerations

After saving the cleaned file, the dataset was loaded into Weka via the Explorer tab. This interface allows interactive testing of the Apriori algorithm, with immediate feedback when parameters are adjusted.

The choice of CSV as the export format was strategic. Rather than converting the dataset back to .arff or another intermediate format, exporting directly from Pandas reduced the number of steps and kept the pipeline transparent and easy to reproduce.

The logical sequence of the preparation stage can be summarized as: **load** → **clean** → **filter** → **remove** → **export**. This sequence forms the foundation of the data preparation phase, which is often underestimated when attention focuses solely on the final rule output.

5. Apriori Configuration in Weka

Apriori is a classic association rule algorithm. It discovers relationships of the form "if a set of items appears, then another item tends to appear alongside it." In a supermarket context, this means identifying recurring co-purchase patterns between products.

The initial run with Weka's default parameters produced highly repetitive rules, concentrated almost entirely around bread and cake. Although mathematically valid, these rules did not offer the diversity required for a meaningful business analysis. Parameter tuning was therefore necessary.

5.1 First run: relaxed confidence

Reducing the minimum confidence threshold to 0.70 broadened the search beyond rules with near-certain outcomes. This change moved the focus away from the bread-and-cake dominance and surfaced rules related to produce, dairy, and breakfast products.

Parameter	Value
minSupport	0.15
minConfidence	0.70
numRules	10

Table 1 – Weka parameters for the first Apriori run.

5.2 Second run: Lift-focused search

The second run was designed to uncover less obvious associations. The minimum support was reduced to 0.05, allowing lower-frequency patterns to surface. The optimization target was changed from Confidence to Lift, shifting the focus from raw rule certainty to the strength of association relative to chance. Increasing the number of rules to 30 revealed additional patterns, including the monthly stock-up profile described in Section 6.3.

Parameter	Value

minSupport	0.05
metricType	Lift
minMetric	1.5
numRules	30

Table 2 – Weka parameters for the second Apriori run.

6. In-Depth Analysis of Selected Rules

After the exploratory phase, three rules with distinct profiles were selected to cover different business scenarios. Each rule is discussed not merely as an algorithmic output, but as a starting point for understanding consumer behavior, store layout decisions, and promotional actions.

Rule	Antecedent	Consequent	Conf.	Lift	Lev.	Business Context
1	`vegetables=t`	`fruit=t`	0.75	1.16	0.07	Produce, health, cross-merchandising
2	`bread and cake=t`	`milk-cream=t`	0.70	1.10	0.05	Breakfast, snacks, promotional kits
3	`total=high`	`biscuits=t, frozen foods=t, tissues-paper prd=t`	0.44	1.91	0.08	Monthly stock-up, salary cycle, seasonal campaigns

Table 3 – Summary of selected rules and their business interpretation.

6.1 Rule 1 – Vegetables imply fruit

The rule vegetables=t → fruit=t represents an intuitive and commercially valuable pattern. When a customer purchases vegetables, there is a 75% probability they will also purchase fruit. This association aligns with the behavior of health-conscious shoppers, those following dietary routines, or customers preparing specific recipes.

The most plausible scenario is a consumer navigating the produce section in search of ingredients for a salad or a balanced meal. The physical proximity of fruits and vegetables in-store also facilitates this type of combined purchase.

A Lift of 1.16 confirms that this co-occurrence exceeds what would be expected under statistical independence. The Leverage of 0.07 indicates the combination appears 7 percentage points more frequently than chance predicts — suggesting genuine purchase intent rather than coincidence.

Retail application: cross-merchandising by co-locating produce sections and displaying recipe suggestions that combine both categories. A salad station featuring vegetables, fruits, and complementary condiments reduces friction and facilitates the buying decision.

6.2 Rule 2 – Bread and cake imply milk and cream

The rule bread and cake=t → milk-cream=t captures a routine breakfast or snack purchase pattern. Bread and cake serve as anchor products, with milk or cream as a natural complement — whether for immediate consumption or home preparation.

In many households, this behavior is tied to the start of the day, afternoon snacks, or the assembly of quick family meals. The customer may not be purchasing a single item in isolation, but composing a small domestic consumption kit.

With a Lift of 1.10 and Leverage of 0.05, the association is moderate but consistent across the dataset. Retail application: promotional bundles such as discounts on butter or cream when bread is purchased increase the average ticket without appearing intrusive, as the offer aligns with a real and recurring consumer need.

6.3 Rule 3 – The monthly stock-up purchase

The rule total=high → biscuits=t, frozen foods=t, tissues-paper prd=t is the most strategically significant of the three. A high transaction total indicates a large basket spanning multiple store sections. This pattern is consistent with monthly restocking behavior, typically observed at the beginning of a pay cycle.

The customer fills the cart with staple foods, frozen products, household items, and paper goods, forming a comprehensive supply run. The Lift of 1.91 — the highest among the selected rules — confirms that this combination is strongly associated beyond chance.

Retail application: seasonal campaigns timed to early-month spending peaks, featuring bundled discounts across the identified categories. This rule is particularly valuable because it connects financial behavior with purchase behavior, creating a clear opportunity for targeted promotions at a predictable moment in the consumer cycle.

7. Metric Interpretation: Support, Confidence, Lift and Leverage

The four metrics function as different lenses on the same rule. Support measures frequency; confidence measures conditional probability; Lift measures association strength relative to chance; Leverage measures the absolute excess of co-occurrence.

Support indicates whether a rule has real relevance in the dataset. Rules with very low support may be interesting outliers, but they rarely justify operational decisions. Confidence is intuitive — it approximates the question "if the customer bought this, what is the probability they also bought that?" — but can be misleading when the consequent item is already very common in the dataset, independent of any rule.

Lift corrects for this limitation by comparing the rule against the expected co-occurrence under independence. A Lift of 1.0 means the rule adds no information beyond chance. A Lift above 1.0 indicates a positive association — the higher the value, the stronger the link between antecedent and consequent.

Leverage works with the difference between observed and expected co-occurrence, making it particularly useful for assessing the absolute impact of an association in terms of purchase volume.

7.1 Leverage in practical terms

A useful way to understand Leverage is to imagine what would happen if the products were purchased entirely at random. If items A and B were statistically independent, there would be a predictable expected frequency of their co-occurrence based solely on their individual frequencies. Leverage measures the difference between what was actually observed and that expected value.

In the vegetables and fruit example, the Leverage of 0.07 indicates that this combination occurs 7 percentage points more frequently than statistical independence would predict. This excess suggests real purchase intent — not random coincidence — and supports the case for targeted retail interventions.

7.2 Reference formulas

Metric	Formula
Support (A → B)	`freq(A ∪ B) / N`
Confidence (A → B)	`supp(A ∩ B) / supp(A)`
Lift (A → B)	`supp(A ∩ B) / [supp(A) × supp(B)]`
Leverage (A → B)	`supp(A ∩ B) − [supp(A) × supp(B)]`
Conviction (A → B)	`[1 − supp(B)] / [1 − conf(A → B)]`

Table 4 – Evaluation metric formulas for association rules.

8. Business Discussion, Limitations and Best Practices

The results obtained demonstrate that association rules serve more than the purpose of identifying co-purchased products. They help frame the store as an interconnected consumption system: produce connects with health and recipes; bakery connects with dairy and snacks; high-value purchases connect with monthly restocking.

These patterns suggest a range of interventions. Some are physical, such as co-locating related products. Others are promotional, such as bundles and targeted discounts. Others are seasonal, such as early-month campaigns timed to salary cycles and increased purchasing power.

An important caveat is that association does not imply causation. The fact that two products appear together does not mean one causes the other to be purchased. What exists is a statistically relevant correlation useful for planning, which should nonetheless be validated by the business team before any operational decision is made.

A further best practice is not to select rules solely on the basis of high metric values. It is necessary to verify that the rule makes commercial sense, that it is not redundant, and that it supports a clear and communicable narrative for any presentation audience. This is precisely why parameter tuning

was as important as the final output.

9. Conclusion

This project executed a complete data mining cycle, from reading the original dataset to interpreting association rules in practical terms. The pipeline included cleaning the data in Python, correctly exporting to CSV, carefully configuring Apriori in Weka, and performing a detailed analysis of the discovered associations.

By varying parameters and exploring different metrics, the analysis moved beyond obvious patterns and surfaced richer rules — including produce habits, snack routines, and the monthly stock-up profile — making the discussion more strategically relevant from a retail perspective.

The core academic contribution lies in the articulation between technique and interpretation. Support, confidence, Lift, and Leverage are not merely numbers: they tell a story about consumer behavior and about how a store can adapt its layout and promotions to align with those patterns.

This report documents not only what was done, but why each decision was made — from the filtering logic to the algorithm configuration to the proposed business interventions. This reasoning strengthens the technical portfolio and the ability to defend methodological choices to any professional audience.

Appendices

Appendix A – Annotated Python Code

The following code represents the complete data preparation pipeline, from reading the .arff file to exporting the cleaned CSV for use in Weka.

```
from scipy.io.arff import loadarff
import numpy as np
import pandas as pd

# Load the .arff file; metadata is discarded as it is not used
data, _ = loadarff('supermarket.arff')

# Convert to NumPy array, then to DataFrame
# dtype=str prevents unintended automatic type conversions
df = pd.DataFrame(np.array(data), dtype=str)
print(f"Loaded: {df.shape[0]} rows, {df.shape[1]} columns")

# Automatically remove all columns prefixed with 'department'
dept_cols = df.filter(like='department').columns
df = df.drop(columns=dept_cols)
print(f"Removed {len(dept_cols)} department columns. Remaining: {df.shape[1]}")

# Remove zero-support columns (all values are '?')
empty_cols = df.columns[(df == '?').all()]
df = df.drop(columns=empty_cols)
print(f"Removed {len(empty_cols)} zero-support columns. Final shape: {df.shape}")
assert not (df == '?').all().any(), "Zero-support columns still present after
cleaning."

# Export without index to preserve data semantics in Weka
output_path = 'supermarket_clean.csv'
df.to_csv(output_path, index=False)
print(f"Exported to {output_path} — {df.shape[0]} rows, {df.shape[1]} columns")
```

Appendix B – Quick Reference for the Oral Defense

Key questions to be prepared to answer:

- Explain why the .arff file needed cleaning before running Weka.
- Walk through the logic of filter(like='department').columns and the drop operation.
- Explain the boolean mask for zero-support columns: (df == '?').all().
- Justify the use of index=False in the CSV export.
- Explain why confidence was reduced from 0.90 to 0.70.
- Explain why switching to Lift as the optimization metric found different and less obvious rules.
- For each selected rule: present the scenario, the proposed intervention, and interpret each metric.

- Explain clearly the difference between Lift and Leverage.
- Reinforce that association is not causation, but is highly actionable for retail planning.

Appendix C – Extended Cleaning Logic

Data cleaning was the most critical step in this project because it established the conditions for Apriori to find reliable and meaningful rules. Removing department columns eliminated structural attributes with no analytical value. Removing zero-support columns prevented noise and reduced dimensionality. Exporting without the Pandas index preserved the semantic integrity of the data in Weka.

When these three steps are performed carefully, the remainder of the analysis becomes cleaner and more coherent. In data mining, thorough data preparation often contributes as much to the quality of results as the algorithm itself. The choice of vectorized Pandas operations — instead of explicit Python loops — also demonstrates command of the toolset and produces code that is both more readable and more reproducible.